

FastAPI

FastAPI - a modern Python framework for building HTTP APIs and other ASGI applications.

Setup new project and add fastapi

```
uv init awesome-project --bare
cd awesome-project
uv add "fastapi[standard]"
```

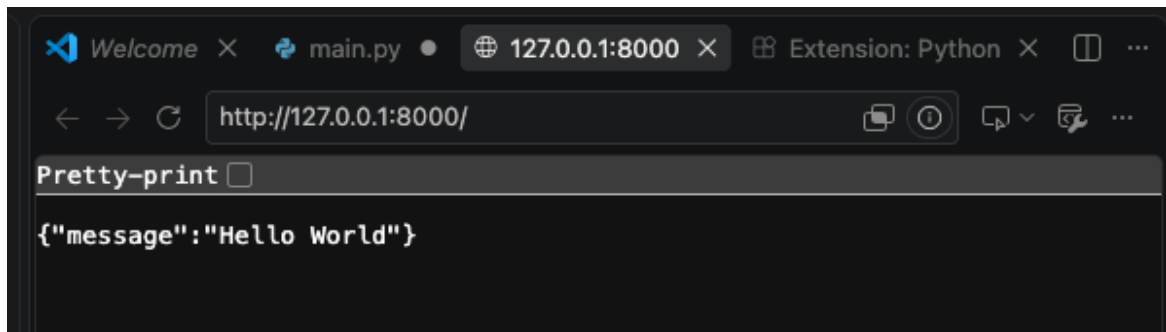
`uv add` creates the project's virtual environment in `.venv`, adds FastAPI to `pyproject.toml`, and creates `uv.lock` so the same package versions can be installed later.

```
main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello World"}
```

```
uv run fastapi dev "main.py" - starts a live server on the file
```



adding /docs to the url allows you to access the apis swagger page, where you can test inputs/outputs

Different Operations

- `POST` : to create data.
- `GET` : to read data.
- `PUT` : to update data.
- `DELETE` : to delete data

`@app.get("/")` -path operation decorator

`async def root():` -path operation function

```
//the value of the parameter will be passes to the function a  
n an argument
```

```
@app.get("/items/{item_id}")  
async def read_item(item_id):
```

```
//you can attach types to those parameters int, str, float, b  
ool
```

```
async def read_item(item_id: int):
```

```
//order is important, if the paths are too similar me can be interpreted as user_id
```

```
@app.get("/users/me")
async def read_user_me():
    return {"user_id": "the current user"}
```

```
@app.get("/users/{user_id}")
async def read_user(user_id: str):
    return {"user_id": user_id}
```

Predefined Values - control input

```
from enum import Enum
```

```
from fastapi import FastAPI
```

```
//only options for input are these three since the argument is of type ModelName
```

```
class ModelName(str, Enum):
    alexnet = "alexnet"
    resnet = "resnet"
    lenet = "lenet"
```

```
app = FastAPI()
```

```
@app.get("/models/{model_name}")
async def get_model(model_name: ModelName):
    //comparison by member
    if model_name is ModelName.alexnet:
        return {"model_name": model_name, "message": "Deep Learning FTW!"}
```

```

        //comparison by value
        if model_name.value == "lenet":
            return {"model_name": model_name, "message": "LeCNN a
ll the images"}

        return {"model_name": model_name, "message": "Have some r
esiduals"}

```

Pathception

```

from fastapi import FastAPI

app = FastAPI()

//you can convert paths if you input is trying to access a ce
rtain path
@app.get("/files/{file_path:path}")
async def read_file(file_path: str):
    return {"file_path": file_path}

```

Query Parameters

```

from fastapi import FastAPI

app = FastAPI()

fake_items_db = [{"item_name": "Foo"}, {"item_name": "Bar"},
{"item_name": "Baz"}]

@app.get("/items/")
async def read_item(skip: int = 0, limit: int = 10):
    return fake_items_db[skip : skip + limit]

```

```
// the url will look something like .../items/?skip=0&limit=10
```

```
// they are query parameters and they can decide how to go about querying a database
```

```
// and since they had "type = num" they are default and would always be within the /items/ path. make a parameter optional by doing "limit: int | None = None"
```

```
// in the order "requirement parameters, defaults, optional
```

```
@app.get("/items/{item_id}")
async def read_user_item(
    item_id: str, needy: str, skip: int = 0, limit: int | None = None
):
    item = {"item_id": item_id, "needy": needy, "skip": skip, "limit": limit}
    return item
```

Request Body

```
from fastapi import FastAPI
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
```

```
app = FastAPI()
```

```
// the post will allow you to create items using the class Item structure, in the above case name and price are the only required variable while the others are optional
```

```
@app.post("/items/")
```

```
async def create_item(item: Item):  
    return item
```

POST

/items/ Create Item

Cancel

Reset

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "name": "bruh",
  "description": "nah",
  "price": 22,
  "tax": 2
}
```

Execute

Clear

Responses

Curly

```
curl -X 'POST' \
  'http://127.0.0.1:8000/items/' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "bruh",
    "description": "nah",
    "price": 22,
    "tax": 2
  }'
```

Request URL

http://127.0.0.1:8000/items/

Server response

Code	Details
200	Response body <pre>{ "name": "bruh", "description": "nah", "price": 22, "tax": 2 }</pre> Download

Response headers

Request body, path, & query parameters

```

from fastapi import FastAPI
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

app = FastAPI()

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item, q: str | None = None):
    result = {"item_id": item_id, **item.model_dump()}
    if q:
        result.update({"q": q})
    return result

/*
If the parameter is also declared in the path, it will be used as a path parameter.

If the parameter is of a singular type (like int, float, str, bool, etc) it will be interpreted as a query parameter.

If the parameter is declared to be of the type of a Pydantic model, it will be interpreted as a request body.
*/

```

Validations

```

from typing import Annotated

from fastapi import FastAPI, Query

app = FastAPI()

// in this case q cannot exceed 50 characters
@app.get("/items/")
async def read_items(q: Annotated[str | None, Query(max_length=50)] = None):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:
        results.update({"q": q})
    return results

```

The two lines below mean the same thing, it can either be type str or none but by default it will be of type None

```

q: str | None = None
q: Annotated[str | None] = None

```

Validations allow for users of the API to better understand the limitations of input and see the errors in their inputs - in the below case I was able to see why my string wasn't retrieving anything.

```

@app.get("/items/")
async def read_items(
    q: Annotated[str | None, Query(min_length=3, max_length=50)] = None,
):
    results = {"items": [{"item_id": "Foo"}, {"item_id": "Bar"}]}
    if q:

```

```
    results.update({"q": q})
return results
```

[illegible]